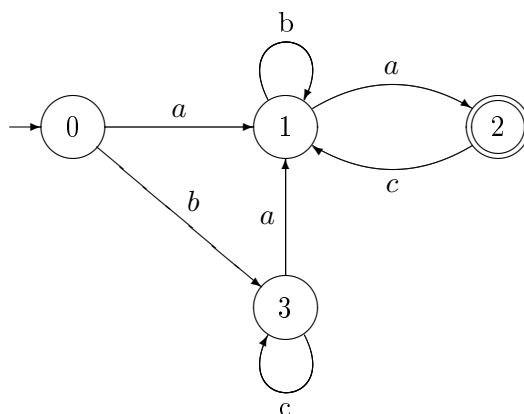


Devoir surveillé

Durée 1h30
(Corrections)

Exercice 1 (Méthode de Brzozowski et McCluskey)

Soit l'automate suivant :



1°) Donnez la définition formelle de cet automate.

Solution : (0,5 point) La définition formelle de cet automate (qui est un AFD) $M = \{Q, \mathcal{A}, \delta, s, F\}$ avec :

— $Q = \{0, 1, 2, 3\}$

— $\mathcal{A} = \{a, b, c\}$

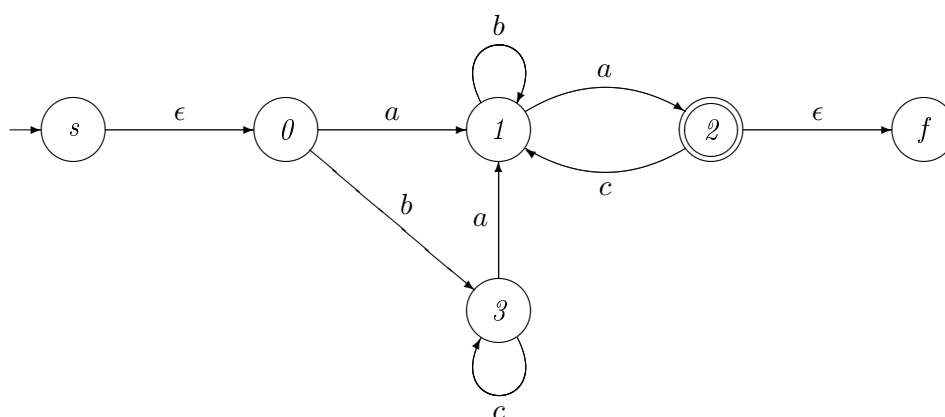
— $\delta =$

q	u	$\delta(q, u)$
0	a	1
0	b	3
1	a	2
1	b	1
2	c	1
3	a	1
3	c	3

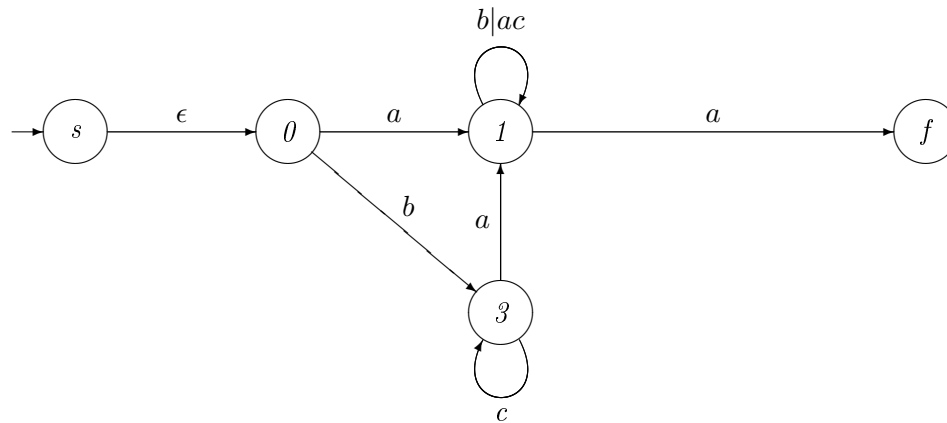
— $s = 0$ et $F = \{2\}$

2°) À l'aide de la méthode de Brzozowski et McCluskey, déterminez l'expression régulière qui représente le langage reconnu par cet automate. Vous indiquerez chaque étape en éliminant les états dans l'ordre 2, 3, 0 et 1.

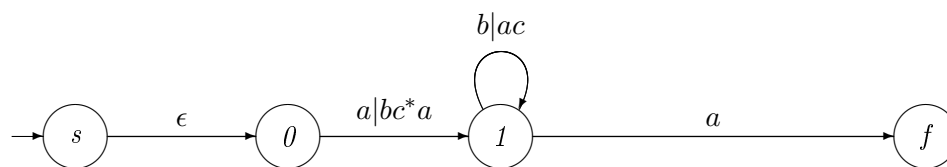
Solution : (3 points) L'automate normalisé :



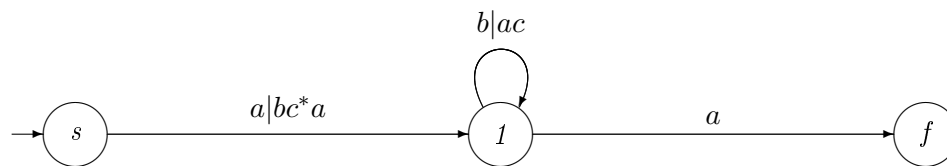
Avec la suppression de l'état 2 :



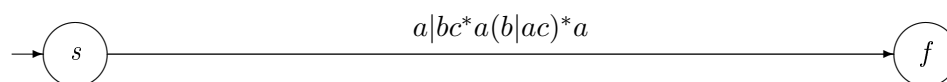
Avec la suppression de l'état 3 :



Avec la suppression de l'état 0 :



Avec la suppression de l'état 1 :



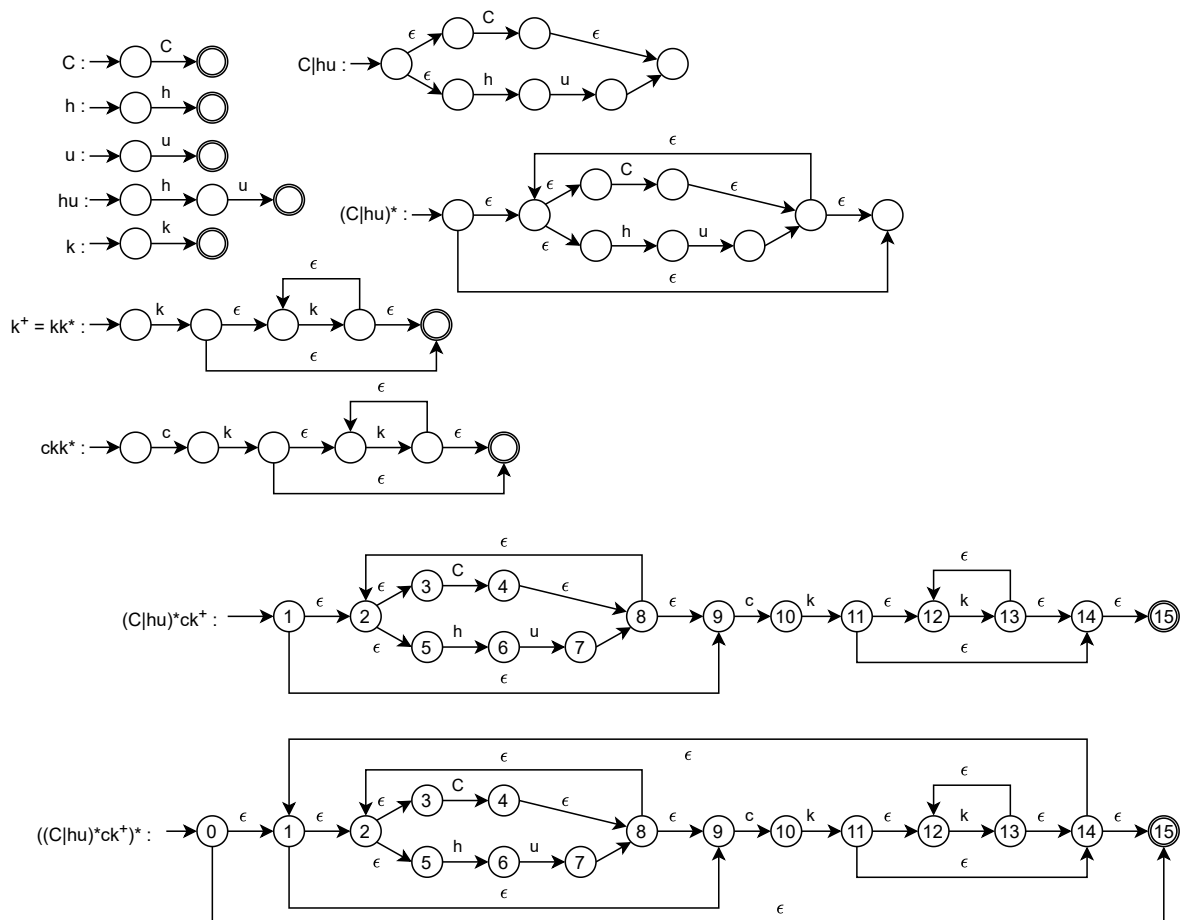
Exercice 2 (Construction de Thomson)

Soit l'expression régulière suivante :

$$((C|hu)^*ck^+)^*$$

1°) En utilisant la construction de Thomson, donnez l'AFN qui reconnaît cette expression.

Solution : (2 points) Voici l'automate obtenu avec les étapes intermédiaires :

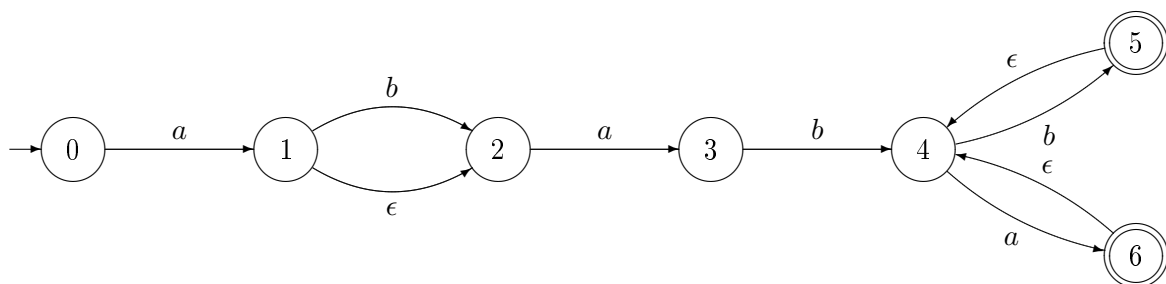


2°) Faites plaisir à Mr. Norris et donnez les dérivations successives pour l'acceptation du mot « Chuck ».

Solution : (1 point) $(q_0, Chuck) \vdash (q_1, Chuck) \vdash (q_2, Chuck) \vdash (q_3, Chuck) \vdash (q_4, huck) \vdash (q_8, huck) \vdash (q_2, huck) \vdash (q_5, huck) \vdash (q_6, uck) \vdash (q_7, ck) \vdash (q_8, ck) \vdash (q_9, ck) \vdash (q_{10}, k) \vdash (q_{11}, \epsilon) \vdash (q_{14}, \epsilon) \vdash (q_{15}, \epsilon)$

Exercice 3 (Détermination)

Soit l'automate suivant :

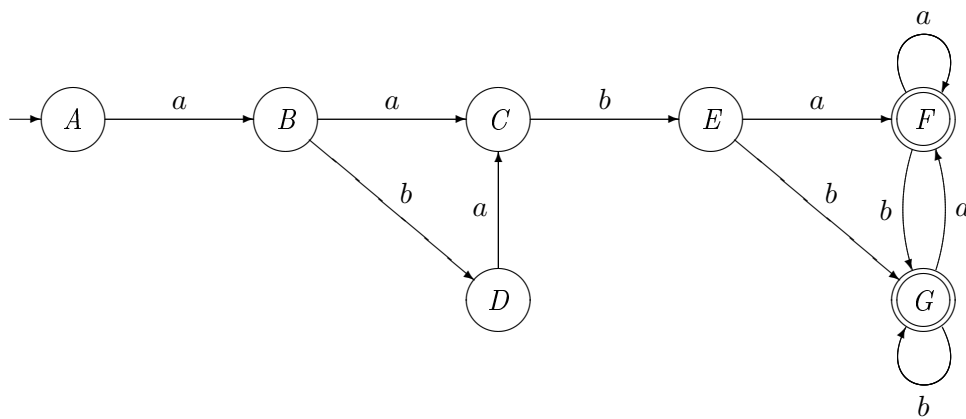


En utilisant l'algorithme présenté en cours, transformez-le en AFD.

Solution : (4 points) Voici l'exécution de l'algorithme :

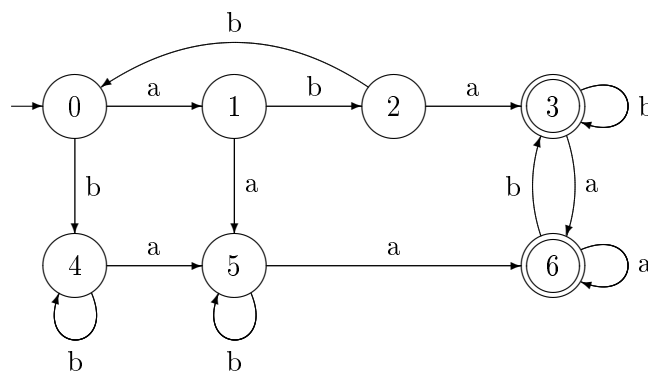
x	$transiter$	y	δ
$A = \{0\}$	$A \xrightarrow{a} \{1\}$	$\{1, 2\} = B$	$A \xrightarrow{a} B$
	$A \xrightarrow{b} \emptyset$		$A \xrightarrow{b} \emptyset$
$B = \{1, 2\}$	$B \xrightarrow{a} \{3\}$	$\{3\} = C$	$B \xrightarrow{a} C$
	$B \xrightarrow{b} \{2\}$		$B \xrightarrow{b} D$
$C = \{3\}$	$C \xrightarrow{a} \emptyset$	$\{2\} = D$	$C \xrightarrow{a} \emptyset$
	$C \xrightarrow{b} \{4\}$		$C \xrightarrow{b} E$
$D = \{2\}$	$D \xrightarrow{a} \{3\}$	$\{3\} = C$	$D \xrightarrow{a} C$
	$D \xrightarrow{b} \emptyset$		$D \xrightarrow{b} \emptyset$
$E = \{4\}$	$E \xrightarrow{a} \{6\}$	$\{4, 6\} = F$	$E \xrightarrow{a} F$
	$E \xrightarrow{b} \{5\}$		$E \xrightarrow{b} G$
$F = \{4, 6\}$	$F \xrightarrow{a} \{6\}$	$\{4, 6\} = F$	$F \xrightarrow{a} F$
	$F \xrightarrow{b} \{5\}$		$F \xrightarrow{b} G$
$G = \{4, 5\}$	$F \xrightarrow{a} \{6\}$	$\{4, 6\} = F$	$G \xrightarrow{a} F$
	$F \xrightarrow{b} \{5\}$		$G \xrightarrow{b} G$

Nous obtenons l'automate suivant :



Exercice 4 (Minimisation)

Soit l'automate suivant :

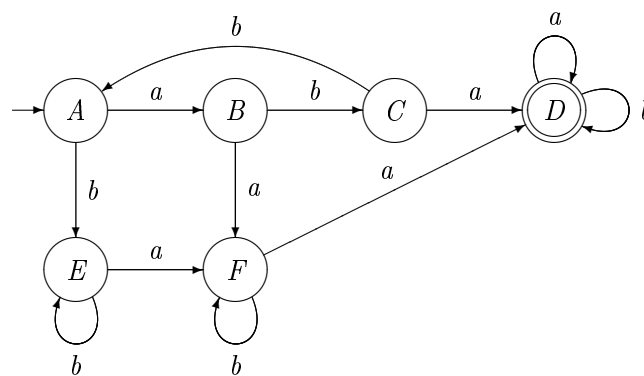


1°) En utilisant l'algorithme présenté en cours, minimisez-le.

Solution : (3,5 points) Voici l'exécution de l'algorithme :

	0	1	2	3	4	5	6
π'	A	A	A	B	A	A	B
$\xrightarrow{a}$	A	A	B	B	A	B	B
$\xrightarrow{b}$	A	A	A	B	A	A	B
π'	A	A	B	C	A	B	C
$\xrightarrow{a}$	A	B	C	C	B	C	C
$\xrightarrow{b}$	A	B	A	C	A	B	C
π'	A	B	C	D	E	F	D
$\xrightarrow{a}$	B	F	D	D	F	D	D
$\xrightarrow{b}$	E	C	A	D	E	F	D
π'	A	B	C	D	E	F	D

Les deux derniers π' sont identiques, l'algorithme s'arrête donc. Voici l'automate obtenu (seuls les états 3 et 6 ont été fusionnés) :



2°) Que peut-on dire sur l'état/les états accepteur(s) de l'automate obtenu ?

Solution : (0,5 point) Il s'agit d'un état puits : aucune transition ne permet de quitter cet état.

Exercice 5 (Automate à pile)

Soit le langage suivant :

$$L = \{a^n b^n c^m d^m \mid n > 0, m \geq 0\}$$

1°) Donnez la définition formelle de l'automate à pile (acceptation sur pile vide) permettant de reconnaître ce langage.

Solution : (4 points) Voici la définition formelle de l'automate $M = \{Q, \mathcal{A}, \mathcal{B}, \Delta, z, s\}$:

- $Q = \{q_0, q_1, q_2, q_3, q_4\}$
- $\mathcal{A} = \{a, b, c, d\}$ et $\mathcal{B} = \{a, c, Z\}$
 - (q_0, a, Z) (q_0, a) On empile le premier 'a'
 - (q_0, a, a) (q_0, aa) On empile les 'a' suivants
 - (q_0, b, a) (q_1, ϵ) Au premier 'b', on passe en q_1
 - (q_1, b, a) (q_1, ϵ) On lit tous les 'b' suivants
 - (q_1, c, ϵ) (q_2, c) On empile le premier 'c'
 - (q_2, c, c) (q_2, cc) On empile les 'c' suivants
 - (q_2, d, c) (q_3, ϵ) On lit le premier 'd'
 - (q_3, d, c) (q_3, ϵ) On lit les 'd' suivants
- $z = Z$ (pour éviter d'accepter le mot vide)
- $s = q_0$

2°) Donnez l'exécution sur le mot $aabbcd$.

Solution : (0,5 point) Voici une exécution :

$$(q_0, aabbcd, Z) \vdash (q_0, abbcd, a) \vdash (q_0, bbcd, aa) \vdash (q_1, bcd, a) \vdash (q_1, cd, \epsilon) \vdash (q_2, d, c) \vdash (q_3, \epsilon, \epsilon)$$

Exercice 6 (Équivalence d'expressions régulières)

Avec R et S deux expressions régulières, nous considérons l'équivalence suivante :

$$(R|S)^* \equiv R^*|S^*$$

Est-elle correcte ?

Solution : (1 point) Supposons que l'alphabet sur lequel sont définies R et S soit $\mathcal{A} = \{a, b\}$ et que les langages reconnus par R et S soient $\mathcal{L}(R) = \{a\}$ et $\mathcal{L}(S) = \{b\}$.

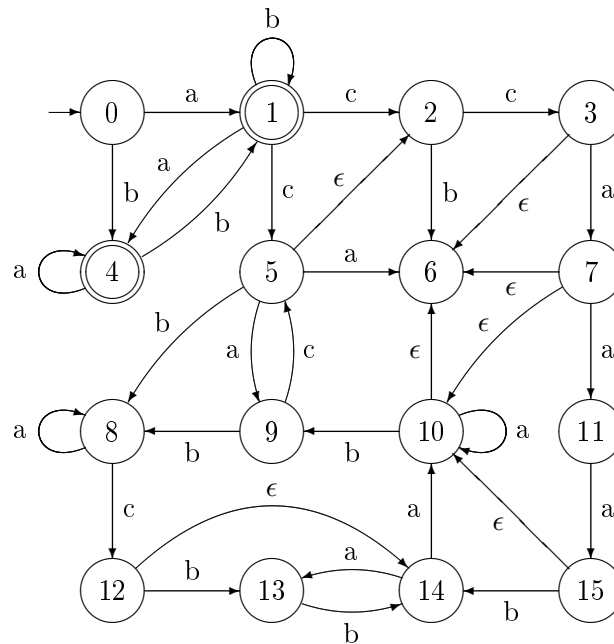
$\mathcal{L}((R|S)^*) = \mathcal{L}((R|S))^* = (\mathcal{L}(R) \cup \mathcal{L}(S))^*$. Ce langage contient le mot vide et toutes les combinaisons des mots de R et de S . Donc $ab \in \mathcal{L}((R|S)^*)$.

R^* est constitué des mots contenant uniquement des a et du mot vide, et S^* est constitué des mots contenant uniquement des b et du mot vide. Or, $\mathcal{L}(R^*|S^*) = \mathcal{L}(R^*) \cup \mathcal{L}(S^*)$. Donc les mots de $\mathcal{L}(R^*|S^*)$ appartiennent soit à $\mathcal{L}(R^*)$, soit à $\mathcal{L}(S^*)$. Donc $ab \notin \mathcal{L}(R^*|S^*)$.

L'équivalence est donc incorrecte.

Exercice 7 (Exercice bonus)

Donnez un AFD et un AFD minimal équivalents à l'automate suivant :



Solution : (2 points) On remarque que si on a un 'c' en entrée lorsqu'on est dans l'état 1, il n'est plus possible de revenir sur les états accepteurs. Donc, seuls les états 0, 1 et 4 sont nécessaires. En supprimant tous les états superflus, nous obtenons alors un AFD.

On observe également qu'une fois un 'a' ou un 'b' rencontré, nous passons dans les états terminaux 1 et 4. Une fois sur ces états, si on rencontre un 'a' ou un 'b', on reste sur les états terminaux. Le langage accepté est donc $(a|b)(a|b)^*$. Donc on peut proposer l'automate minimale suivant :

